

Lecturer

Prof. Dr. Isabel Wagner (<mailto:isabel.wagner@unibas.ch>) Office 05.009

Assistants

Shiva Parsarad (<mailto:shiva.parsarad@unibas.ch>)

Valentyna Pavliv (<mailto:valentyna.pavliv@unibas.ch>)

Release: 4 May 2026

Deadline: 19 May 2026, 23:55 (ADAM)

Requirements

In this exercise, you will create and analyze **differentially private machine learning models**. You will work with the COVID-19 patient pre-condition dataset published by the Mexican government¹. The dataset, translated to English, is available for download at <https://www.kaggle.com/datasets/tanmoyx/covid19-patient-precondition-dataset>, and also included with the instruction files on ADAM (Source: Dirección General de Epidemiología (DGE) de la Secretaría de Salud, SSA/DGE/SINAVE 2025.)

Make sure that you familiarize yourself with the dataset, in particular with the meanings and data types of all columns, before you start working on the tasks.

Requirements common to all tasks:

- (a) Your implementations must be in Python 3.
- (b) You must include your implementation in the provided skeleton `e4.py`, such that we can run `python3 e4.py` and `pytest e4.py`. Jupyter notebooks are NOT allowed.
- (c) The only import statements you may use are:

```
import numpy as np
import pandas as pd
import datetime as dt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score
import seaborn as sns
from scipy import stats
import matplotlib.pyplot as plt
import pytest
```

- (d) Create a brief manual that explains how to run your code and interpret the output. If you do not provide this manual, you will not get any points.
- (e) Write test cases to show the correctness of your implementation.
 - Test cases must use `pytest` (see <https://docs.pytest.org/en/stable/>)
 - Your test cases must have high test coverage, i.e., they must test your entire implementation including edge cases.

¹<https://www.gob.mx/salud/documentos/datos-abiertos-152127>

- Your manual must include the rationale for your test case selection. Why these test cases and not others? Argue why your selected test cases provide good test coverage.

The solutions for this exercise must be delivered in a zip or tar file containing:

- All source codes containing the solutions.
- A **single PDF** file containing the written answers and the manual.

1 Non-private logistic regression (3 points)

Write code to:

- Prepare the data for training, in particular:
 - Remove the *id* column
 - Replace all 97, 98, and 99 values with NaN
 - Replace all 2 values with 0 and all 3 values with 2
 - Convert the *entry_date* and *date_symptoms* columns from dates into the number of days since January 1, 2020
 - Convert the *date_died* column into a 0 if the patient survived (date 9999-99-99) and 1 if they died
- To prepare for logistic regression, impute the missing values, e.g., using the *SimpleImputer* from *sklearn.impute*
- Split the dataframe: variable *y* holds the labels (*date_died* column), variable *X* holds the features (all other columns)
- Create a train/test split so that 20% of all rows are used for testing
- Train a logistic regression model
- Output the accuracy of the trained model

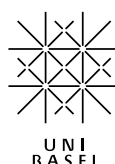
2 Gradient Descent with Renyi Differential Privacy (6 points)

Write code to:

- Implement the function `rdpgd(iterations, alpha, epsilon_bar)`, a variant of gradient descent that:
 - is NOT stochastic, i.e., uses all training samples in each iteration
 - is noisy by using Rényi differential privacy (RDP).
- Your solution should have a **total** privacy cost of $(\alpha, \bar{\epsilon})$ -RDP.
- You can use the following function to compute the gradient of the logistic loss function:

```
# This is the gradient of the logistic loss
def gradient(weights, xi, yi):
    exponent = yi * (xi.dot(weights))
    return - (yi*xi) / (1+np.exp(exponent))
```

Explain:



- Argue why the total privacy cost of your solution is $(\alpha, \bar{\epsilon})$. Use sequential composition.
- Compare the accuracy of your model to the accuracy of non-private logistic regression from Question 1.

3 Differentially Private Stochastic Gradient Descent (10 points)

Write code to:

- Implement differentially private stochastic gradient descent with Gaussian noise in the function `dpsgd(iterations, epsilon, delta, learning_rate, batch_size, C)`. The batch size and clipping threshold C are given as parameters of the function. You can use the global variables `X_train` and `y_train` as feature vectors and output labels.
- Evaluate the privacy/utility trade-off for DPSGD by experimenting with different values of ϵ , number of iterations, learning rate, and batch size, to study the effect of these parameters on the test accuracy of the resulting model.
- Visualize your results in one or more appropriate plots.
- Compute the total privacy cost based on the Moments accountant for your recommended parameter setting.

Explain:

- Describe what you can observe in your plots.
- Based on your observations, which recommendations can you give to set suitable parameter values for this dataset and classification task?

4 Performance evaluation (6 points)

Compare the run-time performance and utility (accuracy) for `rdpgd` and `dpsgd` with $\epsilon = 1$, $\delta = 10^{-5}$ (and equivalent privacy settings for RDP) for different batch sizes and numbers of iterations.

- Visualize your results in one or more appropriate plots and describe your observations. How does each parameter influence the utility and computational performance of the two algorithms?
- Explain the reason for the difference in computation time between `rdpgd` and `dpsgd`.

Hint: To reduce your overall computation time, try to minimize how often you evaluate `rdpgd`.

Hint2: If you think it would improve your answers, you can choose different sets of iteration numbers for the two algorithms.